

Forecast Accuracy Is Not Control Quality: Energy-Aware Benchmarking of Cellular Traffic Forecasters

Colin Minini, Tariq Barki, Alexios Aravanis
CentraleSupélec, Paris-Saclay University

Abstract—Time series forecasting is a key enabler for proactive network management, supporting resource provisioning, energy-aware operation, and service-level compliance. In mobile networks evolving toward 6G, safe operating points depend on an upper bound on traffic that is tight enough to avoid wasteful over-provisioning yet safe enough to avoid violations. We study a shift-aware forecast-to-control pipeline on the public TIM Milan dataset in which a probabilistic forecaster emits median and upper paths, a change-point detector localizes the next regime break, and the controller commits to a safe ceiling only over the predicted pre-break interval. Evaluation uses committed-interval coverage together with a decomposition of sharpness into covered slack, undercoverage frequency, and undercoverage severity. Two empirical findings follow. *First*, DeepAR attains the second-best RMSE yet breaches its safe ceiling on 30.5% of test windows; compared with a pinball-trained LSTM that shares the same recurrent backbone, this indicates that the Gaussian-NLL head, not the backbone, drives the control failure. *Second*, under annual deployment accounting that charges quarterly retraining across markets and live inference every 10 minutes, deep models do not justify their energy cost against Seasonal Naive on this benchmark: zero-shot Chronos-2 is effectively tied on covered sharpness while costing several orders of magnitude more compute, and TFT is the only trained deep model with a meaningful control gain, bought at a comparable operational premium. The benchmark shows that forecast quality does not imply control quality, and that operational compute is not recovered by downstream control utility on this dataset.

Index Terms—probabilistic forecasting, pinball loss, foundation models, change-point detection, energy-aware ML, network control, TIM Milan, 6G

I. INTRODUCTION

Time series forecasting is a key enabler for proactive network management, supporting resource provisioning, energy-aware operation, and service-level compliance. In mobile networks evolving toward 6G, traffic demand progresses through regimes separated by distribution shifts — daily cycles, event surges, and broader demand trends — so that a controller reacting only to current load risks unsafe or wasteful operating points when the next regime arrives [1]–[3]. The relevant control quantity in this setting is not a point estimate of next-step traffic; it is an upper bound on traffic over the next stable sub-interval, tight enough to avoid wasteful over-provisioning yet safe enough to avoid service-level violations.

Classical statistical models such as ARIMA [4] and exponential smoothing [5] provide interpretable baselines but assume stationarity, limiting their ability to capture non-stationary cellular traffic. Deep learning has advanced forecast

accuracy: recurrent architectures such as LSTMs, attention-based models like the Temporal Fusion Transformer (TFT) [6], and autoregressive probabilistic models like DeepAR [7] demonstrate strong results on telecom traffic tasks [8]. More recently, foundation models trained on large diverse corpora — Chronos [9], Chronos-2 [10] — can be deployed zero-shot and bypass periodic retraining entirely.

Despite these advances, existing work addresses the forecasting-to-control problem in parts. Traffic forecasting papers optimize accuracy metrics — RMSE, pinball loss [11], coverage [12] — but stop before the control decision [1], [13]. Network control papers set provisioning or sleeping policies around predicted load levels without explicitly modeling when the next distribution break will occur [14]–[16]. A natural reading of such benchmarks is that a more accurate forecaster yields a better control decision, and that continually retrained or large pre-trained deep models must therefore be worth their energy budget relative to a trivial baseline. This paper argues, with a single dataset and a single pipeline, that both assumptions fail on the same benchmark: *forecast accuracy is not control quality*, and deep-learning forecasters, whether continually retrained or deployed zero-shot, do not clearly beat Seasonal Naive once operational compute is charged explicitly.

We instrument a shift-aware forecast-to-control pipeline — probabilistic forecast $\rightarrow$ break-point detection on the median $\rightarrow$ pre-break safe ceiling from the upper quantile $\rightarrow$ refresh at the predicted break — and evaluate coverage and sharpness on the committed interval $[1, \tau_{\text{pred}}]$. Sharpness is decomposed into covered slack and undercoverage frequency and severity. DeepAR’s $q_{0.99}$ path is recovered in closed form as $\mu + z_{0.99}\sigma$ from a deterministic Gaussian rollout. The upper-quantile target is 0.99, native to the Chronos-2 output grid.

Contributions.

- **Shift-aware forecast-to-control pipeline with committed-interval metrics.** A receding-horizon control loop scored on $[1, \tau_{\text{pred}}]$ with sharpness decomposed into covered slack $\tilde{S}_{\text{cov}}$, undercoverage frequency U_f , and severity U_s .
- **DeepAR shows why forecast quality and control quality diverge.** DeepAR attains the second-best RMSE on TIM Milan yet breaches its safe ceiling on 30.5% of test windows. Compared with a pinball-trained LSTM that shares the same recurrent backbone, the case study points to the Gaussian-NLL head, not the backbone, as the source of the

control failure.

- **Deep learning does not justify its operational energy budget on this benchmark.** Under annual deployment accounting that charges one model per market, quarterly retraining for trainable models, and live inference at the native 10-minute cadence, zero-shot Chronos-2 is effectively tied with Seasonal Naive on covered sharpness while costing 3.03×10^{17} vs 1.14×10^{11} annual FLOPs in the 10-market scenario. TFT is the only trained deep model with a meaningful control gain, and it does so at 1.42×10^{17} annual FLOPs.

The benchmark tables and figures in this paper are generated from the released experiment pipeline and artifacts for the 200-cell TIM Milan subset.

The remainder of the paper is organised as follows. Section II reviews quantile regression, probabilistic forecasters, and change-point detection. Section III presents the shift-aware forecast-to-control pipeline together with its receding-horizon algorithm and committed-interval metric decomposition. Section IV details the five forecasters. Section V describes the TIM Milan benchmark and the operational energy-accounting methodology. Section VI reports the two empirical claims. Section VII concludes and outlines future work.

II. BACKGROUND AND RELATED WORKS

A. Quantile regression and pinball loss

For a target quantile level $q \in (0, 1)$, the pinball loss [11] of a prediction $\hat{y}$ against a truth y is

$$\rho_q(y, \hat{y}) = \begin{cases} q(y - \hat{y}) & \text{if } y \geq \hat{y}, \\ (1 - q)(\hat{y} - y) & \text{if } y < \hat{y}. \end{cases} \quad (1)$$

The empirical risk minimiser of $\mathbb{E}[\rho_q(Y, \hat{Y}(X))]$ is the conditional q -quantile of $Y \mid X$ [11]. Pinball loss is asymmetric, robust, and distribution-free: it makes no parametric assumption about the conditional density. By contrast, Gaussian negative log-likelihood (Gaussian NLL),

$$\mathcal{L}_{\text{NLL}}(y; \mu, \sigma) = \frac{1}{2} \log(2\pi\sigma^2) + \frac{(y - \mu)^2}{2\sigma^2}, \quad (2)$$

recovers quantiles only through the Gaussian inverse CDF and therefore inherits Gaussian tail behaviour. On heavy-tailed targets — and 10-minute cellular traffic is visibly positively skewed — the Gaussian-fit upper quantiles are systematically too narrow. We use this head-loss mismatch as the cleanest explanation for the system-level separation observed in Section VI.

B. Probabilistic forecasters for telecom traffic

Recurrent networks (LSTM [17]) and attention-based encoders (TFT [6]) have been adopted as strong probabilistic forecasters for telecom traffic [8], [13]. DeepAR [7] combines an autoregressive LSTM with a Gaussian likelihood head, recovering quantiles through the Gaussian inverse CDF of its stepwise marginals. Foundation models for time series — Chronos [9], Chronos-2 [10] — are pre-trained on heterogeneous corpora and expose a native quantile output grid, making

them attractive for 6G deployments where periodic retraining is operationally costly. The Seasonal Naive baseline forwards the lag- s value ($s = 144$ steps = one daily cycle here) plus a residual-quantile margin and provides the rational lower bound on the energy–quality Pareto.

C. Change-point detection and CLASP

The pipeline reduces the forecast path to a single control object: the next regime boundary. For this we use CLASP [18], a non-parametric detector that reports the highest-scoring split only when the score exceeds a threshold, and otherwise emits an explicit no-break decision. This is preferable to parametric segmentation in our setting because the controller must distinguish between true regime shifts and stable horizons where the ceiling should cover the full interval.

D. Proper scoring, coverage, and sharpness

Evaluating a probabilistic forecaster at the tail alone is not sufficient: a trivially wide interval maximises coverage at the cost of over-provisioning, and a trivially narrow interval maximises sharpness at the cost of catastrophic violations. Proper scoring rules [19] formalise the tradeoff; coverage and sharpness are the two scalar summaries typically reported [12]. In our pipeline the relevant evaluation interval is not the full horizon but the pre-break sub-interval the controller commits to, $[1, \tau_{\text{pred}}]$. Section III gives the full semantics and a decomposition that separates over-provisioning slack on covered windows from the frequency and severity of ceiling breaches.

E. Energy-aware machine learning

Training and inference cost have become first-class concerns in machine learning research [20], [21]. For 6G control the question sharpens: a forecaster invoked every τ_{pred} across thousands of cells accumulates an inference bill that may dwarf its one-time training cost, and whether that recurring spend buys a useful improvement over a zero-cost heuristic is the operator’s actual decision. We adopt the standard analytical convention [22] — forward FLOPs $\approx 2 \cdot \text{params} \cdot \text{tokens}$, backward $\approx 2 \times$ forward — and price FLOPs under two hardware-scenario J/FLOP constants so that the qualitative ordering is invariant to hardware choice.

III. SHIFT-AWARE FORECAST-TO-CONTROL PIPELINE

A. Design philosophy

The pipeline is intentionally simple. Its value is in making the next predicted distribution break a first-class control object: where existing policies use forecasts to set a resource budget over the full horizon, this policy commits only to the next stable sub-interval and refreshes when that interval ends. Any calibrated forecaster exposing a median path $q_{0.5}(1 : H)$ and an upper path $q_{0.99}(1 : H)$ plugs into the shared interface. The five forecasters of Section IV differ in architecture, training regime, and how they produce uncertainty estimates.

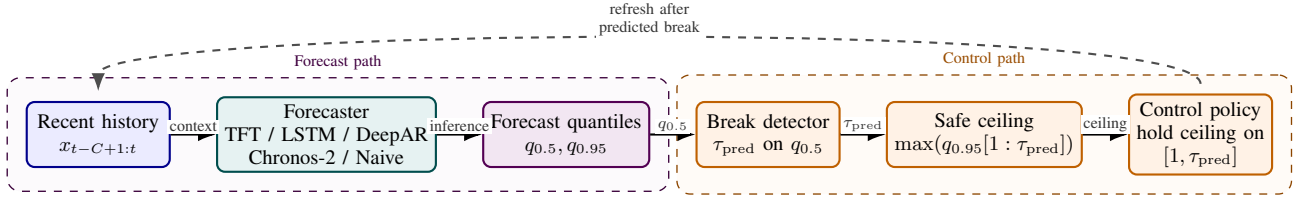


Fig. 1: Shift-aware forecast-to-control pipeline. Any forecaster that produces a median path $q_{0.5}$ and an upper path $q_{0.99}$ over horizon H plugs into the shared interface. The five forecasters evaluated in this paper — TFT, LSTM, DeepAR, Chronos-2, Seasonal Naive — are interchangeable blocks at the left. The downstream controller and evaluation logic are identical across all models.

Algorithm 1 Shift-aware receding-horizon control loop

Require: traffic stream x , context length C , horizon H , forecaster F , change-point detector $\text{CP}(\cdot)$

```

1:  $t \leftarrow C$ 
2: while new observations arrive do
3:    $\mathbf{x}_{\text{ctx}} \leftarrow x_{t-C+1:t}$ 
4:    $(q_{0.5}(1:H), q_{0.99}(1:H)) \leftarrow F(\mathbf{x}_{\text{ctx}})$ 
5:    $\tau_{\text{pred}} \leftarrow \text{CP}(q_{0.5}(1:H))$ 
6:    $c_{\text{safe}} \leftarrow \max_{1 \leq k \leq \tau_{\text{pred}}} q_{0.99}(k)$ 
7:   Provision network at  $c_{\text{safe}}$  for steps  $[t+1, t+\tau_{\text{pred}}]$ 
8:    $t \leftarrow t + \tau_{\text{pred}}$ 
9: end while

```

B. Pipeline stages

Figure 1 shows the full pipeline. Given a traffic context $x_{t-C+1:t}$ of length C :

Stage 1 — Probabilistic forecast. The forecaster produces two paths over the next H steps:

$$q_{0.5}(1:H) \quad \text{and} \quad q_{0.99}(1:H).$$

Stage 2 — Break-point detection. A CLASP change-point detector [18] is applied to the median path $q_{0.5}$. It returns the most prominent predicted break point τ_{pred} — the position that globally maximises the classification score separating the two forecast regimes — or H if no significant break is detected.

Stage 3 — Safe ceiling. The pre-break safe ceiling is

$$c_{\text{safe}} = \max_{1 \leq k \leq \tau_{\text{pred}}} q_{0.99}(k).$$

This ceiling covers the predicted stationary interval using the upper quantile path up to the predicted break.

Stage 4 — Control and refresh. The network is provisioned up to c_{safe} until τ_{pred} . At that point, the pipeline refreshes: it collects new observations, updates the context, and runs stages 1–3 again. This yields a receding-horizon loop driven by predicted regime changes.

Algorithm 1 formalizes the receding-horizon control loop. The refresh is triggered by the predicted break itself: when the detector predicts a nearby change, the controller commits to a shorter ceiling and refreshes sooner; when no break is detected, the ceiling covers the full horizon.

C. Why explicit break detection matters

Restricting the ceiling computation to $[1, \tau_{\text{pred}}]$ separates two objectives that are often conflated: forecast quality over the full horizon and control quality over the next stable interval. A model with good RMSE can still produce a poor control interval if its break point or upper quantile is misplaced — the cautionary tale developed in Section VI.

D. Evaluation metrics on the committed interval

a) *Interval semantics.* The controller commits to c_{safe} over $[1, \tau_{\text{pred}}]$ and then refreshes, so the realised traffic the ceiling has to cover is $\mathcal{I}_{\text{pred}} = x_{1:\tau_{\text{pred}}}$. All interval-based metrics below are evaluated on $\mathcal{I}_{\text{pred}}$, matching what the controller observes before the next refresh. A post-hoc realised interval $\mathcal{I}_{\text{ref}} = x_{1:\tau_{\text{ref}}}$ (with τ_{ref} detected on the realised future path) would over-credit models whose τ_{pred} ran ahead of τ_{ref} .

b) *Coverage and decomposed sharpness.* For window i let $M_i = \max(\mathcal{I}_{\text{pred}}^{(i)})$ denote the realised peak inside the committed interval and let $\Delta_i = c_{\text{safe}}^{(i)} - M_i$ denote the gap between the ceiling and that peak. Coverage is evaluated as a ratio-of-sums over all test windows,

$$\text{Coverage} = \frac{\sum_i \sum_{x \in \mathcal{I}_{\text{pred}}^{(i)}} \mathbf{1}[x \leq c_{\text{safe}}^{(i)}]}{\sum_i |\mathcal{I}_{\text{pred}}^{(i)}|}, \quad (3)$$

$$\tilde{S}_{\text{cov}} = \frac{\sum_{i:\Delta_i \geq 0} \Delta_i}{\sum_{i:\Delta_i \geq 0} M_i} \quad (\text{over-provisioning slack}), \quad (4)$$

$$U_f = \Pr(\Delta_i < 0) = \frac{|\{i : \Delta_i < 0\}|}{|\{i\}|} \quad (\text{undercover. frequency}), \quad (5)$$

$$U_s = \frac{\sum_{i:\Delta_i < 0} (-\Delta_i)}{\sum_{i:\Delta_i < 0} M_i} \quad (\text{undercover. severity}). \quad (6)$$

$\tilde{S}_{\text{cov}}$ is positive by construction and measures over-provisioning on windows the controller actually covers; U_f and U_s measure how often the ceiling breaches and how deep each breach is relative to the realised peak.

c) *Reporting convention.* For the rest of the paper, “coverage” is (3), “relative sharpness (covered)” is $\tilde{S}_{\text{cov}}$, and “undercoverage %” and “undercoverage severity” are U_f and U_s . Ideal behaviour combines high coverage, small positive $\tilde{S}_{\text{cov}}$, and low U_f, U_s .

IV. FORECASTERS

All five forecasters emit the shared $q_{0.5} / q_{0.99}$ interface of Section III over horizon $H = 48$ steps. We detail each family below and specify the closed-form Gaussian quantiles used for DeepAR throughout.

A. Temporal Fusion Transformer (TFT)

TFT [6] combines variable-selection networks that learn which inputs are relevant at each step, an encoder-decoder LSTM (hidden size 128, 2 layers, dropout 0.1), interpretable multi-head self-attention (4 heads, shared value projection, causal mask), gated residual networks for feature enrichment, and a direct quantile head producing $q_{0.5}$ and $q_{0.99}$. Time features — cyclic encodings of hour-of-day and day-of-week — are concatenated with the raw traffic value to form the multi-channel input.

Channel-selective instance normalization. Time features encode calendar position and must not be rescaled. We normalise only the raw-value channel:

$$\text{scale} = \text{mean}(|x_{1:C,0}|) + 1, \quad (7)$$

divide channel 0 by scale before encoding, and rescale the output by scale before computing the loss. The $+1$ prevents division by zero on constant or sparse series. Without this, the validation-to-test RMSE gap exceeds 30% because the test period has different traffic magnitudes than the validation period; with it, the gap closes to under 2%. Training minimises pinball loss at $q \in \{0.5, 0.99\}$; the checkpoint monitor is $\rho_{0.5} + \rho_{0.99}$ on the validation set.

B. LSTM (pinball-head baseline)

The LSTM forecaster uses a 2-layer LSTM (hidden size 128, dropout 0.1) with context-level instance normalization and a direct quantile head that emits all $H \times 2$ outputs in one forward pass. Training minimises pinball loss at $q \in \{0.5, 0.99\}$. This model is included deliberately: it shares its recurrent backbone with DeepAR (211,552 vs 215,202 parameters) but uses a pinball head instead of a Gaussian NLL head, so the LSTM–DeepAR pair isolates the training-loss effect from the architectural effect in the case study of Section VI-A.

C. DeepAR (cautionary baseline, analytic Gaussian)

DeepAR [7] uses an autoregressive LSTM with per-series item embeddings (dimension 20), time features (hour-of-day, day-of-week, log-age), and a Gaussian parameter head that emits (μ_t, σ_t) at each step. Training minimises Gaussian NLL (Eq. 2). As for TFT, the checkpoint monitor is $\rho_{0.5} + \rho_{0.99}$ on validation: Gaussian NLL as monitor triggered early stopping before the quantile paths had matured.

Closed-form Gaussian quantiles. We use a deterministic greedy rollout: at each step the LSTM consumes the *predicted mean* (rescaled) from the previous step, and the head emits the Gaussian marginal (μ_t, σ_t) . Upper-quantile paths are then

$$\hat{q}_\alpha(t) = \mu_t + z_\alpha \sigma_t, \quad z_{0.99} \approx 2.326, \quad (8)$$

TABLE I: TIM Milan benchmark configuration.

Quantity	Value
Complete grid cells	8,883
Trainable subset	200 cells
Total timestamps	8,928
Cadence	10 min
Date range	2013-11-01 to 2014-01-01
Context length C	144 steps (24 h)
Horizon H	48 steps (8 h)
Train / val / test rows	6,249 / 893 / 1,786
Test windows	7,400
Upper-quantile target	$q = 0.99$

with z_α the standard-Gaussian inverse CDF. This is the DeepAR $q_{0.99}$ path used in every forecast, control, and energy result reported below.

D. Chronos-2 (foundation model, zero-shot)

Chronos-2 [10] is a patch-based encoder-only transformer pre-trained on a large, diverse public and synthetic corpus and deployed without any Milan-specific fine-tuning. A single forward pass emits 21 quantile levels, including $q_{0.99}$ natively, so no interpolation is required at our target quantile. In our setup, Chronos-2 is the foundation-model entrant: broadly trained once, deployed directly into the control loop, and free of periodic retraining. We use the `amazon/chronos-2` checkpoint (120 M parameters).

E. Seasonal Naive (training-free baseline)

The Seasonal Naive forecaster requires no training. The median forecast is the lag-144 value (one daily cycle at the 10-minute cadence): $q_{0.5}(k) = x_{t-144+k}$. The upper path is the median plus the 0.99-quantile of the historical residual distribution $\{x_\tau - x_{\tau-144}\}$ measured over the context window. Despite its simplicity, Seasonal Naive is not a straw man: in Section VI-B it anchors the operational energy-sharpness frontier, and the energy comparison asks what, if anything, each trained or zero-shot deep model buys over this free baseline.

V. EXPERIMENTAL SETUP

A. TIM Milan benchmark

The benchmark is built from the TIM Milan telecom activity dataset released by Barlacchi *et al.* [23]. We use only the `InternetTraffic` field at the original 10-minute cadence, retaining only fully observed cells. The complete-cell pool contains 8,883 cells across 8,928 timestamps from 2013-11-01 through 2014-01-01.

We evaluate on a trainable subset of 200 cells, selected as the first 200 complete cells by ascending grid-cell identifier — a neutral rule that avoids activity-based cherry-picking. The dataset is split chronologically in 70/10/20 proportion, yielding 6,249 train rows, 893 validation rows, and 1,786 test rows. Evaluation uses non-overlapping horizon-step windows (step = 48) anchored at split start, yielding 7,400 test windows (200 series $\times$ 37 windows), identical across all forecasters.

Context length. $C = 144$ steps spans exactly one full daily cycle at 10-minute cadence. Shorter contexts miss the dominant periodicity; longer contexts risk including multiple regime changes in the conditioning window.

B. Trained model hyperparameters

TFT, LSTM, and DeepAR share the recurrent backbone: two layers, hidden size 128, dropout 0.1. TFT adds 4-head interpretable self-attention with a shared value projection and a causal mask. LSTM and TFT output two quantiles directly at $\{0.5, 0.99\}$. DeepAR outputs Gaussian (μ, σ) and recovers the upper path from the inverse CDF at the same level.

LSTM and TFT are trained with the Adam optimizer and pinball loss [11] at $q \in \{0.5, 0.99\}$. DeepAR uses Gaussian negative log-likelihood (Eq. 2) as its training loss, consistent with its probabilistic output head. The early-stopping monitor for all three models is

$$\mathcal{L}_{\text{monitor}} = \text{pinball}_{0.5} + \text{pinball}_{0.99}$$

evaluated on the validation set. Training runs for a maximum of 30 epochs with early-stopping patience 3, and the checkpoint with the lowest monitor value is retained. At inference, DeepAR uses the deterministic analytic-Gaussian rollout described in Section IV-C: upper quantiles are recovered as $\mu_t + z_\alpha \sigma_t$.

C. Chronos-2

Chronos-2 is evaluated in zero-shot mode using the public `amazon/chronos-2` checkpoint (120M parameters) with no fine-tuning on Milan data. The wrapper exposes a 21-quantile output grid covering $[0.01, 0.99]$ in steps of ~ 0.05 . The pipeline reads index 10 ($q_{0.5}$) and index 20 ($q_{0.99}$) directly. No quantile extrapolation is performed.

A single validation-selected CLASP configuration is shared across all five forecasters: `min_size = 8`, `period_length = 16`, `score_threshold = 0.5`. The detector only sees the median path, so the downstream control logic is identical once each model has produced $q_{0.5}$ and $q_{0.99}$.

D. Metrics

Forecast quality is measured by RMSE on the median path, pinball loss at $q \in \{0.5, 0.99\}$, and $q_{0.99}$ coverage (fraction of realized values below $q_{0.99}$).

System quality is reported on the τ_{pred} -committed interval (Section III): coverage rate on $\mathcal{I}_{\text{pred}}$ (Eq. 3), relative sharpness on covered windows $\tilde{S}_{\text{cov}}$ (Eq. 4), undercoverage-window fraction U_f (Eq. 5), and undercoverage severity U_s (Eq. 6).

E. Energy accounting

We price each forecaster under a simple operational deployment model. First, one live inference call for a single series

is priced under the standard analytical convention [22]:

$$F_{\text{fwd}} \approx 2 \text{ params tokens}, \quad (9)$$

$$F_{\text{train}} \approx 3 F_{\text{fwd}} N_{\text{iters}}, \quad (10)$$

$$N_{\text{calls}}^{\text{series}} = D \cdot \frac{24 \text{ h}}{\text{cadence}}, \quad (11)$$

$$N_{\text{calls}}^{\text{market}} = N_{\text{series}} \cdot N_{\text{calls}}^{\text{series}}, \quad (12)$$

$$F_{\text{ann}}^{(m)} = \mathbf{1}_{\text{trainable}} \cdot F_{\text{train}} \cdot R \cdot m + F_{\text{fwd}} \cdot N_{\text{calls}}^{\text{market}} \cdot m, \quad (13)$$

The $3\times$ factor in training bundles one forward plus a $\approx 2\times$ backward pass [22]. Parameter counts for the trainable models are recovered exactly from the saved checkpoints (TFT 1.83 M, LSTM 211,552, DeepAR 215,202); Chronos-2 uses its public 120M checkpoint count and incurs zero retraining cost because the pre-training bill is not charged to the operator. Seasonal Naive and Chronos-2 therefore pay only the deployment-inference term. DeepAR inference uses the analytic-Gaussian rollout (Section IV-C) at one forward pass per live call.

The deployment assumptions are intentionally simple. One model is trained per market over that market’s 200-series fleet, trainable models retrain quarterly ($R = 4$ cycles/year), and live inference runs every 10 minutes for $D = 365$ days. With the Milan cadence this yields $N_{\text{calls}}^{\text{series}} = 52,560$ forecast calls per year and $N_{\text{calls}}^{\text{market}} = 10,512,000$ calls per year for one 200-series market. We report three market scenarios $m \in \{1, 10, 100\}$ and use $m = 10$ as the headline table/figure scenario in Section VI.

Charging one forecast call per cadence step is a conservative upper bound relative to the receding-horizon loop of Section III, which only invokes the forecaster once per committed interval $[1, \tau_{\text{pred}}]$ and therefore issues fewer calls per year whenever $\tau_{\text{pred}} > 1$. Because the same divisor would apply to every forecaster, the relative ordering of models on the energy axis is invariant under this simplification; the absolute FLOPs figures reported in Section VI are worst-case.

FLOPs are converted to joules under two hardware-scenario constants that bracket plausible deployments: $J_{\text{GPU}} = 1.3 \times 10^{-12}$ J/FLOP (NVIDIA A100 fp16, sustained ≈ 312 TFLOPS at ≈ 400 W) and $J_{\text{CPU}} = 1.0 \times 10^{-9}$ J/FLOP (x86 Skylake fp32, ≈ 1 GFLOP/s/W). These are end-to-end “wall-energy per algebraic FLOP” constants; the qualitative ordering of forecasters on the Pareto plane is invariant under any constant in the plausible range, which is what the $10^3 \times$ GPU-vs-CPU gap is meant to show.

VI. RESULTS AND DISCUSSION

We report on the TIM Milan test split (200 series $\times$ 37 windows = 7,400 windows) using the committed-interval metrics of Section III at the $q_{0.99}$ upper-quantile target. Tables II–IV support the DeepAR case study in Section VI-A and the operational energy comparison in Section VI-B.

TABLE II: Forecast-quality metrics on the TIM Milan test set (7,400 windows). Lower is better for RMSE and pinball; coverage target 0.99.

Model	RMSE _{q0.5}	Pinball _{q0.5}	Pinball _{q0.99}	Cov _{q0.99}
TFT	5.756	1.185	0.112	0.982
LSTM	6.103	1.514	0.129	0.997
DeepAR	5.942	1.489	0.329	0.840
Chronos-2	7.070	2.108	0.190	0.953
Seasonal Naive	7.263	1.405	0.180	0.927

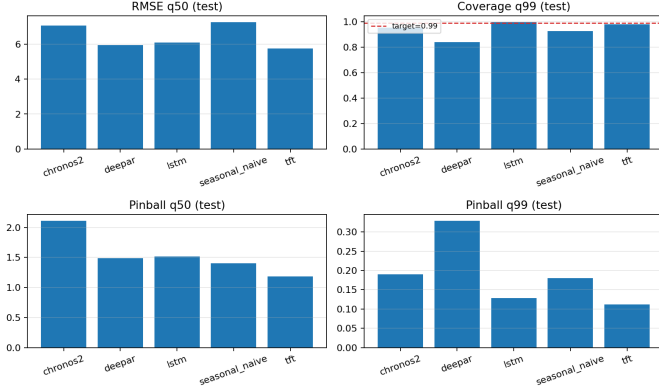


Fig. 2: Forecast metrics on the TIM Milan test set. Left: RMSE and pinball at $q_{0.5}$. Right: pinball at $q_{0.99}$ and coverage. The 0.99 target is shown as a dashed line.

A. DeepAR case study: forecast quality, control failure, and loss choice

On point-accuracy (Table II), TFT leads on RMSE (5.756) and pinball_{q0.5} (1.185), and DeepAR is second on RMSE (5.942) — ahead of the pinball-trained LSTM (6.103) and well ahead of Chronos-2 and Seasonal Naive. If the benchmark stopped here, DeepAR would look like the second-strongest forecaster in the pool.

The control-loop scoreboard (Table III) overturns that ranking. DeepAR breaches its safe ceiling on $U_f = 30.5\%$ of test windows, nearly $8\times$ the TFT rate (4.1%) and $23\times$ the LSTM rate (1.3%); when it breaches, the ceiling sits 10.8% below the realised peak ($U_s = 0.108$). The same tail problem appears on the forecast row: DeepAR’s pinball_{q0.99} is 0.329, nearly $3\times$ LSTM’s 0.129, and its $q_{0.99}$ coverage is 0.840 against a 0.99 target.

The five-panel gallery in Figure 4 makes the failure visible: against the same four test windows the other four forecasters hold a ceiling above the realised peak, while DeepAR’s ceiling often sits below a traffic burst its Gaussian tail does not accommodate. This is the most surprising empirical result from the benchmark: a model can look strong as a forecaster and still be unreliable as a controller.

Tables II and III also include a natural experiment. LSTM and DeepAR share a near-identical recurrent backbone (211,552 vs 215,202 parameters; the $\approx 3,650$ -parameter gap is DeepAR’s item embedding for 200 cells). They differ in

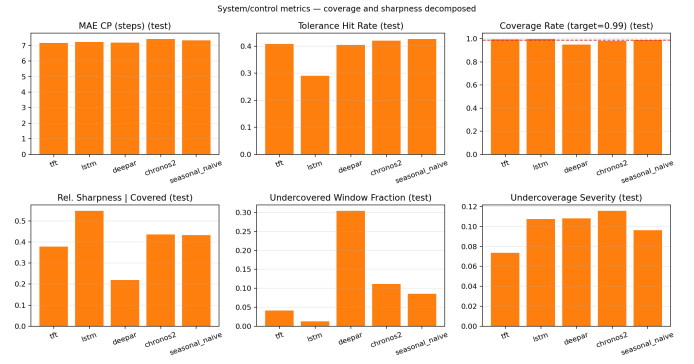


Fig. 3: Six-panel control-loop dashboard on the test set after CLASP break-point detection and safe-ceiling construction at $q_{0.99}$. All interval-based metrics are evaluated on $\mathcal{I}_{\text{pred}}$; the 0.99 coverage target is shown as a dashed line. Note the *decomposed* sharpness panels (covered sharpness, undercoverage frequency, undercoverage severity), which show how DeepAR combines tight covered windows with frequent ceiling breaches.

one substantive respect: LSTM’s direct quantile head is trained with pinball loss (Eq. 1), while DeepAR’s Gaussian parameter head is trained with Gaussian NLL (Eq. 2). The outcome is unambiguous. LSTM, pinball: $U_f = 1.3\%$, coverage 0.999, $\tilde{S}_{\text{cov}} = 0.549$ (the loosest but safely covered ceiling in the pool). DeepAR, Gaussian NLL: $U_f = 30.5\%$, coverage 0.949, $\tilde{S}_{\text{cov}} = 0.219$ on the covered subset. The two forecasters pull the $q_{0.99}$ path from nearly the same recurrent representation; the head changes, and the control behaviour changes with it.

The closed-form DeepAR quantiles from Section IV-C make the diagnosis cleaner. Milan traffic at 10-minute cadence is bursty, positively skewed, and heavy-tailed, so a Gaussian upper quantile recovered as $\mu + z_{0.99}\sigma$ under-shoots the empirical 0.99 tail whenever the realised bursts are heavier than Gaussian. TFT (pinball, different backbone) and Chronos-2 (native quantile grid, zero-shot) stay directionally on the safe side of the control loop, which reinforces the same interpretation. The benchmark therefore attributes the DeepAR failure to the Gaussian-NLL head and its tail assumption, not to the recurrent backbone.

B. Energy vs sharpness: the Pareto for 6G control

Having established in the DeepAR case study that forecast quality and control quality can diverge sharply, we close the benchmark on the operator’s question: *does the recurring energy budget of each forecaster buy a useful improvement over the free baseline?* Table IV and Figure 5 give the answer.

Seasonal Naive anchors the floor. Under annual deployment accounting, one 200-series market requires 10,512,000 live forecast calls per year at the native 10-minute cadence. Seasonal Naive prices this at only 1.14×10^{10} FLOPs per market-year and 1.14×10^{11} FLOPs in the 10-market headline scenario, yet still delivers $\tilde{S}_{\text{cov}} = 0.433$ at $U_f = 8.5\%$ — not

TABLE III: Control-loop metrics on the TIM Milan test set with τ_{pred} -committed interval semantics and decomposed sharpness. Coverage, $\tilde{S}_{\text{cov}}$, U_f , and U_s are defined by Equations (3), (4), (5), and (6). Lower U_f and U_s are better; coverage target ≥ 0.99 .

Model	Coverage	Rel. Sharp. (cov.) $\tilde{S}_{\text{cov}}$	Undercov. win. % U_f	Undercov. severity U_s
TFT	0.996	0.378	4.1%	0.074
LSTM	0.999	0.549	1.3%	0.107
DeepAR	0.949	0.219	30.5%	0.108
Chronos-2	0.981	0.437	11.1%	0.116
Seasonal Naive	0.989	0.433	8.5%	0.096

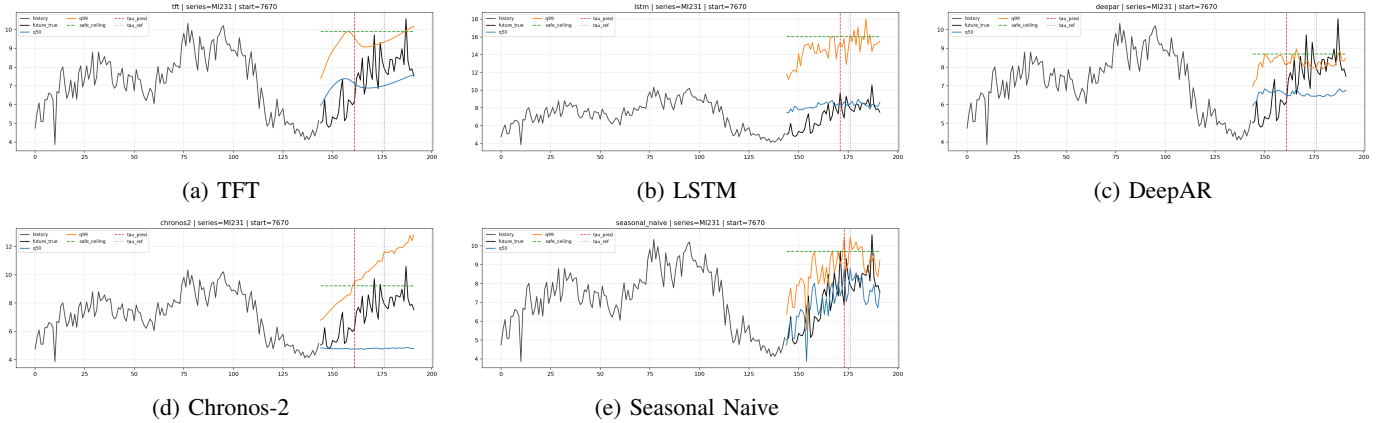


Fig. 4: Committed-interval forecasts on the same four test windows across all five forecasters. In each panel the safe ceiling c_{safe} is flat over $\mathcal{I}_{\text{pred}} = [1, \tau_{\text{pred}}]$ and the realised peak M_i inside the committed interval is marked. DeepAR’s ceiling sits below the realised peak on many windows — the visual signature of the 30.5% undercoverage reported in Table III — while TFT, LSTM, Chronos-2, and Seasonal Naive cover.

TABLE IV: Annual deployment accounting for the headline 10-market scenario using Equations 9–13. One model is trained per market, trainable models retrain quarterly, and live inference runs every 10 minutes over 365 days. “Annual infer / market” isolates recurring production cost for one 200-series market; retrain, total, and joule columns scale to 10 markets. $J_{\text{GPU}} = 1.3 \times 10^{-12}$ J/FLOP, $J_{\text{CPU}} = 1.0 \times 10^{-9}$ J/FLOP. Δ vs Naive is $\tilde{S}_{\text{cov}}^{(\text{naive})} - \tilde{S}_{\text{cov}}^{(\text{model})}$; positive means the model is tighter than the free baseline on covered windows.

Model	Params	Annual infer / market	Annual retrain (10 mkt)	Annual total (10 mkt)	E_{GPU} (10 mkt)	E_{CPU} (10 mkt)	$\tilde{S}_{\text{cov}}$	Δ vs Naive
TFT	1,826,790	7.37e15	6.80e16	1.42e17	1.84e5	1.42e8	0.378	+0.055
LSTM	211,552	8.54e14	2.95e16	3.81e16	4.95e4	3.81e7	0.549	−0.115
DeepAR	215,202	8.69e14	4.81e16	5.68e16	7.38e4	5.68e7	0.219	+0.215
Chronos-2	120,000,000	3.03e16	0	3.03e17	3.94e5	3.03e8	0.437	−0.003
Seasonal Naive	—	1.14e10	0	1.14e11	1.48e-1	1.14e2	0.433	0

the tightest or the most covered, but firmly inside the band of the deep models.

Chronos-2 does not beat Seasonal Naive — even after removing retraining. A 120 M-parameter zero-shot foundation model avoids the seasonal retraining term entirely, but its live inference bill dominates: 3.03×10^{16} FLOPs per market-year and 3.03×10^{17} FLOPs in the 10-market scenario, versus 1.14×10^{10} and 1.14×10^{11} for Seasonal Naive. Yet the control-quality comparison does not reward the extra compute. On covered sharpness Chronos-2 ($\tilde{S}_{\text{cov}} = 0.437$) and Seasonal Naive (0.433) are effectively tied. On the undercoverage decomposition Chronos-2 is strictly *worse*: it breaches the ceiling on $U_f = 11.1\%$ of windows (vs Seasonal Naive’s 8.5%) and deeper when it does ($U_s = 0.116$ vs 0.096). Zero-shot deployment removes retraining, but it does not remove a

large per-call inference bill.

TFT is the only trained DL model that buys a real control gain. TFT combines a large live-inference bill (7.37×10^{15} FLOPs per market-year) with a non-trivial retraining bill (6.80×10^{16} FLOPs across 10 quarterly-retrained markets), reaching 1.42×10^{17} annual FLOPs in the headline scenario. In exchange it delivers $\tilde{S}_{\text{cov}} = 0.378$, a $\Delta = +0.055$ sharpness improvement over the free baseline, with the lowest undercoverage severity in the pool ($U_s = 0.074$). This is the only trained deep-learning point on the table that clearly improves the control objective, but it does so at more than six orders of magnitude above Seasonal Naive’s deployment cost.

LSTM is Pareto-dominated. LSTM reaches 3.81×10^{16} annual FLOPs in the 10-market scenario and still lands at $\tilde{S}_{\text{cov}} = 0.549$, which is looser than both Chronos-2 and

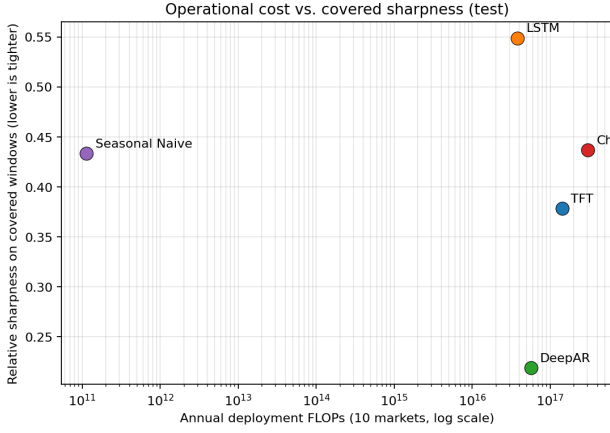


Fig. 5: Operational energy-vs-sharpness Pareto on the test split for the 10-market annual deployment scenario. The x -axis prices quarterly retraining plus live 10-minute inference over 365 days (log scale). The y -axis is $\tilde{S}_{\text{cov}}$ on the predicted committed interval (lower is tighter). The frontier is anchored on the left by Seasonal Naive and on the lower-right by TFT.

Seasonal Naive. Its only win is breach frequency ($U_f = 1.3\%$), which matters if “never breach” dominates every other consideration, but on the energy–slack tradeoff it is dominated.

DeepAR is not a deployable point. Section VI-A already removes DeepAR on U_f ; its $\Delta = +0.215$ is an artefact of conditioning on the 69.5% of windows it covers at all. Its 5.68×10^{16} annual FLOPs (10-market), dominated by quarterly retraining, buys no control-quality win.

The net picture: *no deep-learning forecaster — continually retrained (LSTM, DeepAR, TFT) or zero-shot (Chronos-2) — dominates the one-line seasonal lag once annual deployment cost is charged.* The $m \in \{1, 10, 100\}$ sweep scales totals linearly and preserves the ordering; TFT’s $+0.055$ sharpness gain is the lone exception, earned at six orders of magnitude above Seasonal Naive’s bill.

VII. CONCLUSION

We built a shift-aware forecast-to-control pipeline on TIM Milan, formalized its receding-horizon control loop, and evaluated five probabilistic forecasters — TFT, LSTM, DeepAR, Chronos-2, Seasonal Naive — under committed-interval semantics and a decomposed sharpness metric set. One methodological contribution and two empirical claims close the benchmark.

Method. The controller should be evaluated on the interval it actually commits to, not on a post-hoc interval defined after the fact. Coverage on $\mathcal{I}_{\text{pred}}$, together with the decomposition into covered slack, undercoverage frequency, and undercoverage severity, makes the control objective directly measurable.

DeepAR case study. DeepAR posts the second-best RMSE (5.942), yet its safe ceiling breaches the realised peak on 30.5% of the 7,400 test windows ($U_s = 0.108$). Compared with an LSTM that shares the same recurrent backbone but is

trained with pinball loss, the evidence points to the Gaussian-NLL head as the unit of failure: swapping the head moves undercoverage from 1.3% to 30.5%. The benchmark therefore shows both that forecast quality does not imply control quality and that, for this task, loss/head choice matters more than backbone similarity.

Operational energy. Under annual deployment accounting that charges quarterly retraining across markets and live inference at the native 10-minute cadence, Seasonal Naive remains the floor at 1.14×10^{11} annual FLOPs in the 10-market scenario while still delivering $\tilde{S}_{\text{cov}} = 0.433$ at $U_f = 8.5\%$. The 120 M-parameter zero-shot Chronos-2 does not clear this floor: its retraining term is zero, but its 10-market annual inference bill still reaches 3.03×10^{17} FLOPs, with covered sharpness (0.437) effectively tied with Seasonal Naive and worse undercoverage on both frequency (11.1% vs 8.5%) and severity (0.116 vs 0.096). TFT is the only trained deep model that meaningfully improves on the free baseline ($+0.055$ covered sharpness at 1.42×10^{17} annual FLOPs), while LSTM and DeepAR are Pareto-dominated. On this benchmark, deep models do not earn their operational compute back in control utility over a one-line seasonal lag.

The benchmark is reproducible from the released experiment pipeline and artifacts for the 200-cell TIM Milan subset.

Future work. Two directions follow. (i) **Scale to the full 8,883-cell TIM Milan benchmark and validate on operator-grade sector telemetry.** The question is whether the loss-vs-architecture separation and the deep-learning-vs-naive parity survive at city scale and on production data where traffic patterns are richer than the public proxy. (ii) **Cost-aware per-cell forecaster selection.** The same FLOPs/joule accounting supports allocating forecasters per cell rather than deploying one model fleet-wide: busy or bursty cells where Seasonal Naive breaches most often are candidates for TFT; stable cells stay on the zero-training baseline. The committed-interval decomposition already produces the cell-level signal (U_f , U_s , and their spatial distribution) that such a routing policy would need.

REFERENCES

- [1] E. Lykakis, I. O. Vardiambasis, and E. Kokkinos, “Data traffic prediction for 5g and beyond: Emerging trends, challenges, and future directions: A scoping review,” *Electronics*, vol. 14, no. 23, p. 4611, 2025.
- [2] N. Rajule, M. Venkatesan, R. Menon, and A. Kulkarni, “Network traffic prediction with reduced power consumption towards green cellular networks,” *International Journal of Computer Network and Information Security*, vol. 15, no. 6, pp. 64–77, 2023.
- [3] F. Rau, I. Soto, D. Zabala-Blanco, C. Azurdia-Meza, M. Ijaz, S. Ekpo, and S. Gutierrez, “A novel traffic prediction method using machine learning for energy efficiency in service provider networks,” *Sensors*, vol. 23, no. 11, p. 4997, 2023.
- [4] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015.
- [5] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
- [6] B. Lim, S. O. Arik, N. Loeff, and T. Pfister, “Temporal fusion transformers for interpretable multi-horizon time series forecasting,” *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021.

- [7] D. Salinas, V. Flunkert, and J. Gasthaus, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," *arXiv preprint arXiv:1704.04110*, 2017.
- [8] S. Althamary *et al.*, "Enhanced multi-task traffic forecasting in beyond 5g networks: Leveraging transformer technology and multi-source data fusion," *Future Internet*, vol. 16, no. 5, p. 159, 2024.
- [9] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. P. Arango, S. Kapoor, J. Zschiegner, D. Maddix, M. W. Mahoney, K. Torkkola, A. G. Wilson, M. Bohlke-Schneider, and Y. Wang, "Chronos: Learning the language of time series," *arXiv preprint arXiv:2403.07815*, 2024.
- [10] A. F. Ansari *et al.*, "Chronos-2: From univariate to universal forecasting," *arXiv preprint arXiv:2510.15821*, 2025.
- [11] R. Koenker and G. Bassett, "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.
- [12] P. F. Christoffersen, "Evaluating interval forecasts," *International Economic Review*, vol. 39, no. 4, pp. 841–862, 1998.
- [13] W. Jiang, "Cellular traffic prediction with machine learning: A survey," *Expert Systems with Applications*, vol. 201, p. 117163, 2022.
- [14] Y. Zhu and S. Wang, "Joint traffic prediction and base station sleeping for energy saving in cellular networks," in *IEEE International Conference on Communications*, 2021.
- [15] J. Lin, Y. Chen, H. Zheng, M. Ding, P. Cheng, and L. Hanzo, "A data-driven base station sleeping strategy based on traffic prediction," *IEEE Transactions on Network Science and Engineering*, 2024.
- [16] M. Masoudi *et al.*, "Digital twin assisted risk-aware sleep mode management using deep q-networks," *arXiv preprint arXiv:2208.14380*, 2022.
- [17] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [18] A. Ermschaus, P. Schäfer, and U. Leser, "ClaSP: Parameter-free time series segmentation," in *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM)*, 2022, pp. 3787–3791.
- [19] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.
- [20] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 3645–3650.
- [21] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021.
- [22] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.
- [23] G. Barlacchi, M. D. Nadai, R. Larcher, A. Casella, C. Chitic, G. Torrisi, F. Antonelli, A. Vespignani, A. Pentland, and B. Lepri, "A multi-source dataset of urban life in the city of milan and the province of trentino," *Scientific Data*, vol. 2, p. 150055, 2015.